

Algorithme de génération de labyrinthe

Introduction et Problématique

La génération d'un labyrinthe semblable à un labyrinthe Pac-Man pose un défi algorithmique singulier. En effet, contrairement à la génération d'un labyrinthe standard, ici notre labyrinthe devait respecter un cahier des charges esthétique et fonctionnel strict afin de garantir la jouabilité.

Les fonctionnalités importantes que nous étions contraints de respecter sont les suivantes :

- *Garantir une symétrie axiale* : Un labyrinthe Pac-Man est toujours symétrique selon l'axe vertical, offrant une bonne lisibilité pour le joueur.
- *Garantir une topologie cyclique* : Le graphe ne doit pas être un arbre (c'est-à-dire un graphe sans cycles) car cela créerait des culs-de-sac qui rendent la fuite face aux fantômes impossible. Il fallait donc des boucles.
- *Avoir des zones réservées* : La Ghost House centrale, les murs permanents autour et les tunnels latéraux.

Les algorithmes de génération classique (DFS, Prim ou Kruskal) produisent par défaut des labyrinthes parfaits (arbres couvrants), remplis de culs-de-sac et dépourvus de symétrie.

Pour répondre à ces contraintes contradictoires, j'ai développé une approche hybride en trois phases, combinant masquage structurel, théorie des graphes symétriques et post-traitement topologique.

1. Phase 1 : Modélisation et Gabarit Structurel (Template)

Avant d'appliquer la moindre logique aléatoire, il est impératif de définir le "squelette" immuable du labyrinthe. Nous avons modélisé le labyrinthe via la classe Maze qui représente une grille de cellules typées par l'énumération CellState (MUR, SOL, MUR_PERMANENT, GHOST_HOUSE, TUNNEL, PORTE).

1.1. Le Mur Permanent

Pour empêcher l'algorithme de génération de déstructurer les éléments fixes du gameplay (Ghost House et les bordures), nous avons introduit l'état MUR_PERMANENT. Ce marquage rend ces zones immuables avant même le lancement de l'étape de creusement aléatoire.

1.2. Zones Protégées

Le gabarit assure la présence de trois éléments de gameplay intouchables :

- *La Ghost House* : Une zone de 5x8 cellules au centre est définie comme GHOST_HOUSE et encadrée de murs permanents. Cela garantit que les fantômes ont toujours un point de spawn valide et protégé.

- *Les bordures* : Une double épaisseur de murs sur les contours empêche la génération de sorties accidentelles.
- *Les Tunnels* : Les accès latéraux (pour le wrap-around) sont pré-creusés à mi-hauteur et marqués pour rester toujours ouverts.

1.3. Dégagement des accès

Une simple protection des murs ne suffit pas, il faut aussi garantir la circulation autour de ces zones. J'ai donc implémenté une étape de nettoyage dans le template :

- *Autour de la Ghost House* : Une boucle itérative parcourt toutes les cellules adjacentes à la Ghost House afin de les forcer à l'état SOL. Cela crée un couloir fluide pour assurer une bonne circulation autour de la Ghost House.
- *Entrée des Tunnels* : De la même manière, les zones situées juste devant les entrées des tunnels et sur les côtés du tunnel sont forcées à SOL pour assurer une bonne circulation du Pac-Man à son entrée dans le jeu.

Cette approche permet donc d'abstraire la complexité de ces structures lors de la phase de creusement, l'algorithme de génération de labyrinthe ne se concentre donc que sur les murs destructibles.

2. Phase 2 : L'Algorithme de Kruskal Symétrique

Le cœur de la génération repose sur une variante de l'algorithme de Kruskal. Dans sa version classique, Kruskal sélectionne des arêtes (murs) aléatoires et les ouvre si elles connectent deux ensembles disjoints de cellules.

Afin de satisfaire la contrainte de symétrie axiale, il a fallu redéfinir l'élément fondamental manipulé par l'algorithme.

2.1. Abstraction par "Paires de Murs" (WallPair)

Au lieu de traiter les murs individuellement, l'algorithme manipule des objets WallPair.

Une WallPair est une structure immuable (Java Record) qui encapsule deux murs miroirs :

- Le mur gauche à la position $P(x,y)$.
- Le mur droit symétrique à la position $P'(width - 1 - x, y)$.
- Les cellules adjacentes à ces deux murs.

L'initialisation de l'algorithme consiste à lister toutes les paires de murs possibles en ne parcourant que la moitié gauche de la grille ($x < width / 2$), garantissant ainsi que chaque mur du labyrinthe appartient à une et une seule paire.

2.2. Logique de Fusion (Union-Find)

L'algorithme maintient une structure de données d'ensembles disjoints (sets) pour suivre la connectivité des cellules.

- On mélange aléatoirement la liste des paires de murs (WallPair).
- Pour chaque paire, on examine uniquement le côté gauche : si la cellule C1 et la cellule C2 séparées par le mur gauche appartiennent à des ensembles différents (elles ne sont pas encore connectées) alors on valide l'ouverture.
- L'ouverture est double et atomique : Si la condition est remplie, la méthode carvePassage casse le mur gauche et le mur droit simultanément.
- Les ensembles des cellules concernées sont fusionnés des deux côtés du labyrinthe pour maintenir la cohérence de la carte de connectivité.

Cette méthode garantit mathématiquement la symétrie : il est impossible qu'un chemin existe à gauche sans son équivalent à droite. De plus, comme Kruskal produit un Arbre Couvrant Minimal, nous avons la garantie que toutes les cellules accessibles sont connectées entre elles et donc nous n'avons pas de zones isolées.

3. Phase 3 : Post-traitement topologique : La Création de cycles

A la sortie de la phase 2, nous obtenons un labyrinthe parfait, ce qui veut dire qu'il existe un chemin unique entre deux cellules et le labyrinthe ne possède pas de cycles. Pour un labyrinthe Pac-Man, cela pose un très gros problème car le joueur se retrouverait constamment piégé dans des culs-de-sac sans aucune échappatoire.

J'ai donc implémenté une passe de post-traitement removeDeadEnds() pour ainsi transformer l'arbre obtenu en un graphe cyclique. L'algorithme procède de manière itérative :

- On parcourt la grille pour identifier les culs-de-sac. Un cul-de-sac est défini comme une cellule SOL possédant exactement un seul voisin de type SOL et que des MUR autour.
- Une fois le cul-de-sac identifié, on examine ses murs adjacents. On cherche donc un mur qui, s'il est brisé, déboucherait sur une autre cellule SOL créant ainsi une boucle.
- Enfin, parmi les murs candidats, on en choisit un aléatoirement. Bien évidemment, pour respecter la symétrie du labyrinthe, on brise ce mur ainsi que son symétrique de l'autre côté du labyrinthe.

Cette étape est répétée jusqu'à l'obtention d'un nombre nul de culs-de-sac. Elle permet donc d'éliminer les zones de blocage et assurer une navigation fluide des entités du jeu.

Conclusion et Validation

L'implémentation de cet algorithme hybride a permis d'obtenir une infinité de labyrinthes très ressemblant à un labyrinthe "Pac-Man".

La validation de notre algorithme est assurée par des tests unitaires (MazeTest.java) qui vérifient après chaque génération que la grille n'est pas nulle et que pour tout point (x,y), l'état de la cellule est strictement identique à l'état de la cellule symétrique ($\text{width} - 1 - x, y$).